# Lesson 6 RRC PWM Servo Control Principle and Source Code Analysis

## 1. PWM Servo Description

A servo is a common motor drive device used to control the position, speed, and acceleration in a mechanical system. It typically consists of a motor, a control circuit, and a feedback system.

In terms of type, servos can be categorized into analog servos, PWM (pulse width modulation) servos, and bus servos. This section mainly focuses on PWM servos.

PWM servos use pulse width modulation signals to control the angular position of the servo. In simple terms, a PWM servo controls the angular position by varying the pulse width of the signal. The range of the pulse width determines the extreme angular positions of the servo, while the period of the control signal determines the refresh frequency of the signal. By adjusting the duration of the pulse width, precise control of the servo position can be achieved.

PWM achieves a level state between 0 and 1, meaning that when PWM is output, the voltage can be any value between 0 and 3.3V/5V.

When using PWM control, it is essential to know two parameters of the object being controlled:

Its PWM frequency (corresponding to the timer cycle)

Its operating pulse width (corresponding to the PWM duty cycle)

The control signal enters the signal modulation chip from the receiver channel to obtain a DC bias voltage. Inside, a reference circuit generates a reference signal with a period of 20ms and a width of 1.5ms. The obtained DC bias

voltage is compared with the voltage of the potentiometer to produce a voltage difference output. Finally, the positive and negative outputs of the voltage difference are sent to the motor driver chip to determine the motor's forward and reverse rotation. When the motor speed is constant, the potentiometer is driven to rotate through the cascade reduction gear, ensuring the voltage difference becomes zero, and the motor stops rotating.

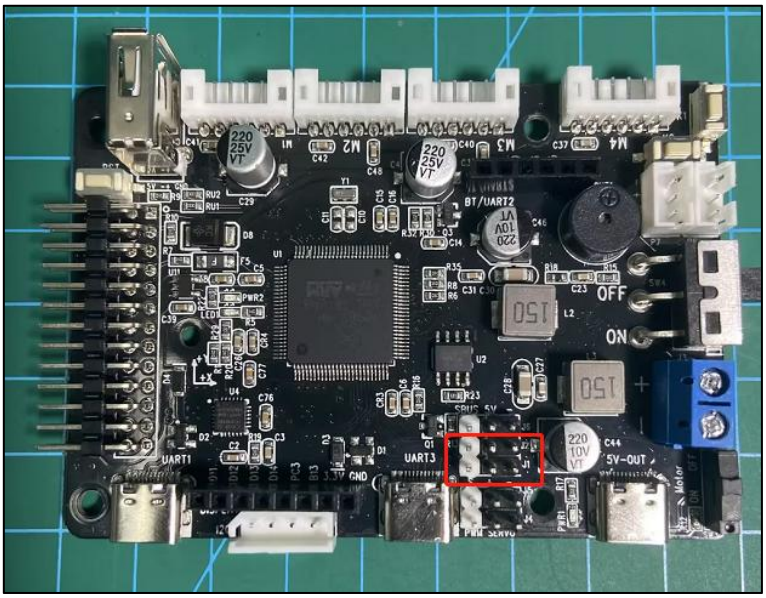
## 2. Wiring Method:

### 2.1 Required Stuffs:

1. stm32 controller
2. LD-1501MG digital servo

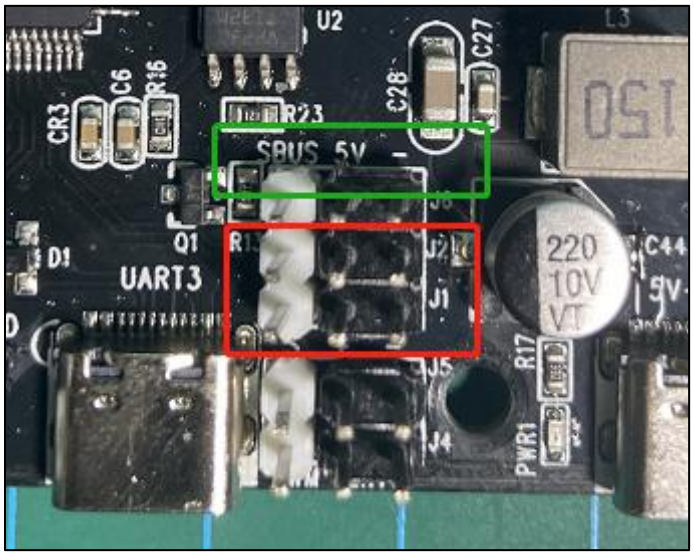
### 2.2 Connection Instructions

1. The diagram below shows the reserved interfaces J1 and J2 provided by the STM32 main control board for the PWM servo.




The enlarged detail image below shows the reserved connection ports J1 and

J2, highlighted in the red box. There are three types of connections: signal line (S), power supply (+5V), and ground (−), as indicated in the green box.



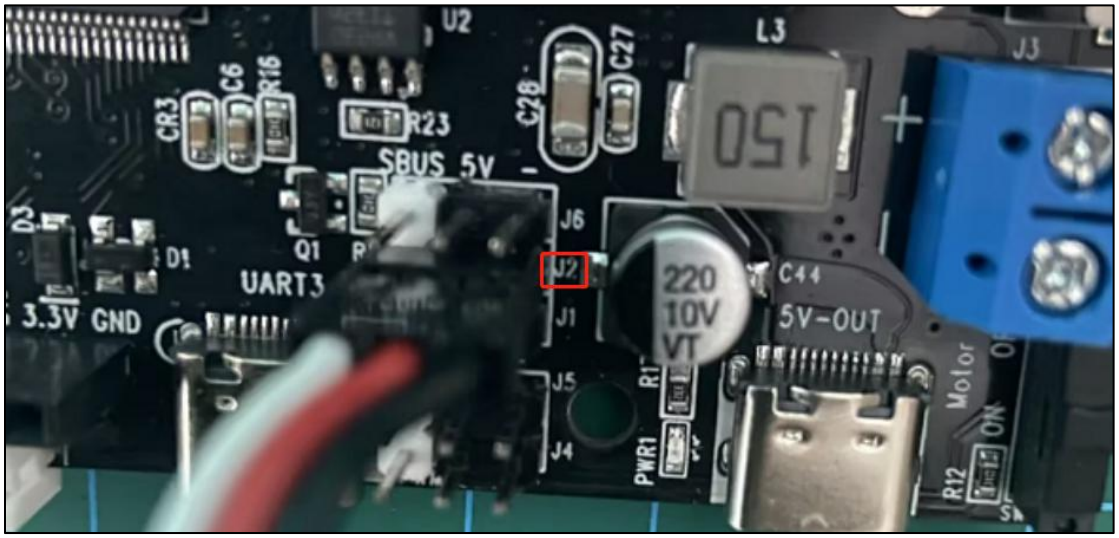

2. Connect the servo's 3-pin cable to the corresponding J2 or J1 interface, as shown below:

The connection method is as follows:

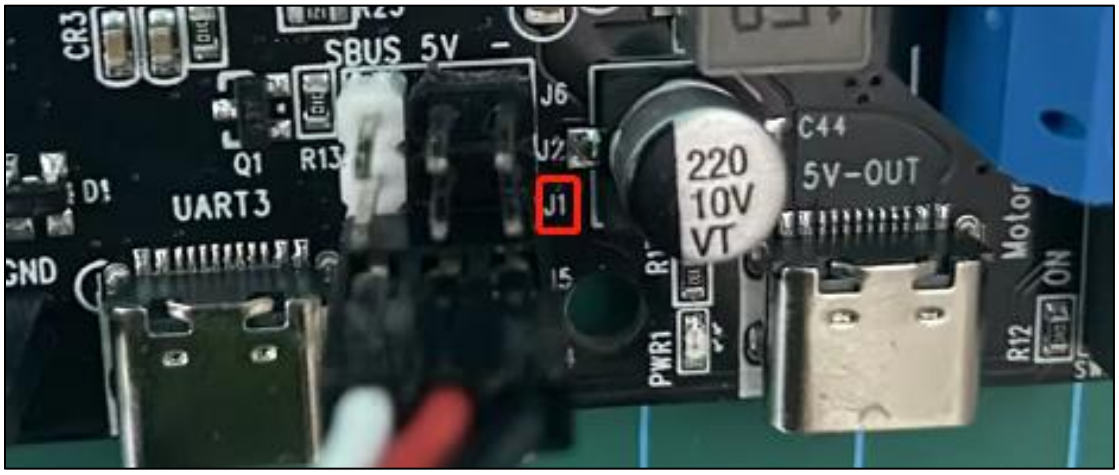
White -> Signal line (S)

Red -> Power supply (+5V)

Black -> Ground (−)




Connect to J2




Connect to J1

The STM32 main control board can be powered by our 7.4V lithium battery pack. For detailed connection instructions, please refer to "Lithium Battery Usage and Precautions" in the current document path.

---

**Attention: The servo motor must be connected according to the positions shown in the document diagram. Avoid connecting it incorrectly or mistakenly to prevent burning out the motor. Our company will not take any responsibility!!!**

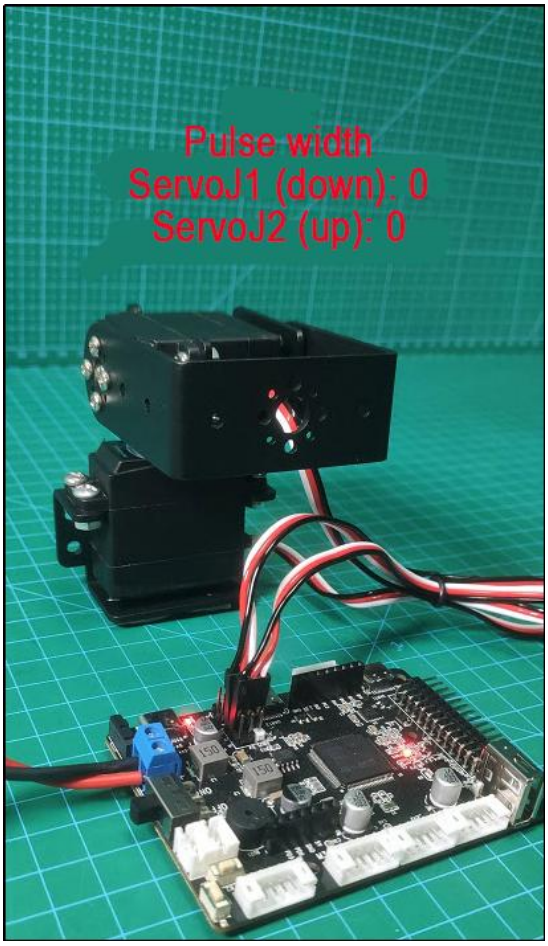
---

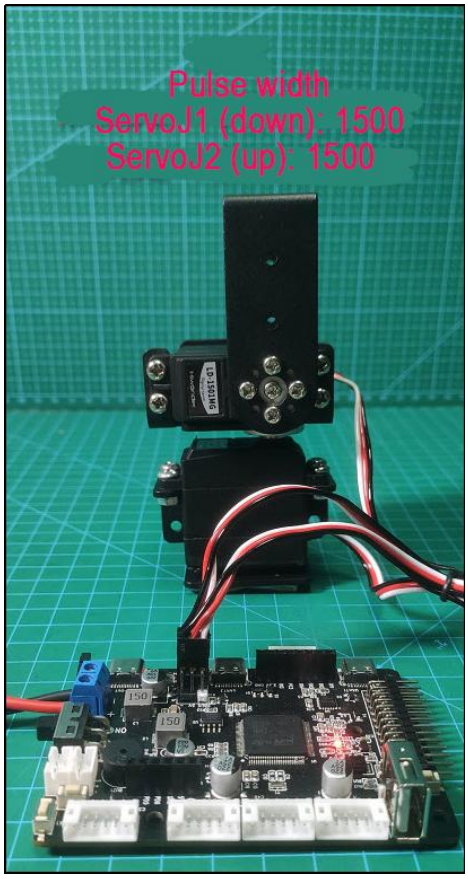
## 3. Program Burning

The sample program can be found in '**Section 6: RRC PWM Servo Control Example/rosrobotcontrollerm4_pwmServo**'. For the steps and precautions required for programming burning, please refer to '2. STM32 Development Fundamentals/Lesson 3 Project Compilation and Download' document. Simply compile the 'rosrobotcontrollerm4_pwmServo' program located in the current document path to generate the hex file and proceed with the burning process.

## 4. Program Outcome

Based on the set PWM servo pulse width values, the servo rotates to the corresponding angles, as shown in the following diagram:




Pulse width
ServoJ1 (down): 1500
ServoJ2 (up): 1500


Pulse width
ServoJ1 (down): 2500
ServoJ2 (up): 2500

# 5. Code Analysis

## 5.1 Header File

### File location: Hiwonder/Peripherals

### Object PWMServoObjectTypeDef:

```
17
18  typedef struct PWMServoObject  PWMServoObjectTypeDef;
19
```

Calling the PWM control object generated by the PWMServoObject structure.

### PWMServoObject Structure:

```
20 /**
21  * @brief PWM舵机对象结构体
22  */
23 struct PWMServoObject {
24     int id;     /**< @brief 舵机ID */
25     int offset; /**< @brief 舵机偏差 */
26     int target_duty;  /**< @brief 目标脉宽 */
27     int current_duty; /**< @brief 当前脉宽 */
28     int duty_raw;     /**< @brief 机器脉宽，最终写入定时器的脉宽，包含了offset， == current_duty + offset */
29
30     /* 速度控制需要的变量 */
31     uint32_t duration;    /**< @brief 舵机从当前角度运动到指定角度的时间，也就是控制速度 单位:ms */
32     float duty_inc; /**< @brief 每次位置更新的脉宽增量 */
33     int  inc_times; /**< @brief 需要递增的次数 */
34     bool is_running;  /**< @brief 舵机是否在转动过程中 */
35     bool duty_changed; /**< @brief 脉宽是否被改变 */
36
37     /* 外部提供，硬件抽象接口 */
38     void (*write_pin)(uint32_t new_state); /* IO口电平设置 */
39 };
```

This structure is used to configure PWM control objects.

Parameters:

Id: Servo ID

offset: Servo offset

target_duty: Target pulse width

current_duty: Current pulse width

duty_raw: Raw pulse width, the width finally written into the timer

Duration: Time for the servo to move from the current angle to the specified angle, controlling the speed

duty_inc: Pulse width increment for each position update

inc_times: Number of increments needed

is_running: Whether the servo is in the process of rotation

duty_changed: Whether the pulse width has been changed

Function write_pin:

This function is used for setting the IO port level. After initialization, this function will be used to set the 0-3 PWM servos.

It has the parameter 'new_state' representing the current pin's high or low level.

**Declare function pwm_servo_object_init:**

```
41 /**
42  * @brief 舵机对象初始化
43  * @param object 要进行初始化舵机对象指针
44  * @retval None.
45 */
46 void pwm_servo_object_init(PWMServoObjectTypeDef *object);
47
```

This function initializes the PWM servo object. Instantiation requires initializing the servo object pointer.

Declare function pwm_servo_duty_compare:

```
49  * @brief 舵机脉宽控制
50  * @detials 计算脉宽变化、当前速度所需要的脉宽并实现控制，需要50ms调用一次
51  * @param self 需要控制的舵机对象指针
52  * @retval None.
53 */
54 void pwm_servo_duty_compare(PWMServoObjectTypeDef *self);
```

This function controls the PWM servo pulse width, calculates the pulse width change and the pulse width required for the current speed, and implements control. It needs to be called every 50ms and requires setting the servo object pointer for control.

Declare function pwm_servo_set_position:

```
/**
 * @brief 设置舵机角度
 * @detials 用于设置舵机的角度，实际只是设置舵机对象的几个内部变量，不会马上实际控制舵机，实际控制舵机是由 pwm_servo_duty_compare来计算并控制
 * @param self 需要控制的舵机对象指针
 * @param 舵机新的脉宽 500~2500间的一个整数值
 * @param None.
 */
void pwm_servo_set_position (PWMServoObjectTypeDef *self, uint32_t duty, uint32_t duration);
```

Set the angle of the PWM servo. Actually, it only sets several internal variables of the servo object and does not immediately control the servo. The actual servo control is calculated and controlled by pwm_servo_duty_compare.

Declare function pwm_servo_set_offset:

```
/**
 * @brief 设置舵机偏差
 * @param self 需要控制的舵机对象指针
 * @param 新的舵机偏差
 * @param None.
 */
void pwm_servo_set_offset(PWMServoObjectTypeDef *self, int offset);
```

Set the offset of the PWM servo, requiring the servo object pointer for control.

## 5.2 Source Code

File location: Hiwonder/Peripherals

Function pwm_servo_duty_compare:

```
void pwm_servo_duty_compare(PWMServoObjectTypeDef *self)   //脉宽变化比较及速度控制
{
    // 根据新设置的目标重新计算舵机控制参数
    if(self->duty_changed) {
        self->duty_changed = false;
        self->inc_times = self->duration / 20; // 计算需要递增的次数
        if(self->target_duty > self->current_duty) { /* 计算总的位置变换量 */
            self->duty_inc = (float)(-(self->target_duty - self->current_duty));
        } else {
            self->duty_inc = (float)(self->current_duty - self->target_duty);
        }
        self->duty_inc /= (float)self->inc_times; /* 计算每控制周期位置增量 */
        self->is_running = true;  // 舵机开始动作
    }

    // 需要控制舵机转动以到达新的位置
    if(self->is_running) {
        --self->inc_times;
        if(self->inc_times == 0) {
            self->current_duty = self->target_duty;   //最后一次递增就直接将设定值赋给当前值，保证最总位置正确
            self->is_running = false; //到达设定位置，舵机停止运动
        } else {
            self->current_duty = self->target_duty + (int)(self->duty_inc * self->inc_times);
        }
    }
    self->duty_raw = self->current_duty + self->offset; // 动作要加上舵机偏差
}
```

This function is used for controlling pulse width change comparison and speed

control.

Parameters: PWMServoObjectTypeDef instantiated object that needs to be controlled.

Firstly, it calculates the control parameters of the servo according to the newly set target. Then, it controls the servo to gradually rotate to the corresponding new position based on the newly set position. Finally, when the last increment occurs, the new position will be assigned to the current position, ensuring the final position is correct.

**Function: pwm_servo_set_position：**

```
43  void pwm_servo_set_position (PWMServoObjectTypeDef *self, uint32_t duty, uint32_t duration)
44  {
45      duration = duration < 20 ? 20 : (duration > 30000 ? 30000 : duration); /* 限制最短/最长运动时间 */
46    duty = duty > 2500 ? 2500 : (duty < 500 ? 500 : duty); /* 限制脉宽最大/最小值 */
47      self->target_duty = duty;
48      self->duration = duration;
49      self->duty_changed = true; /* 标记目标位置发送变换，让 pwm_servo_duty_compare 计算新的运动参数 */
50  }
```

This function is used to set the rotation position of the servo.

Parameters: PWMServoObjectTypeDef instantiated object that needs to be controlled, rotation position (pulse width), required time.

Firstly, it is necessary to limit the duration of the movement as well as the maximum and minimum values of the pulse width. Then, assign the corresponding rotation position and time, as well as the switch to control the servo rotation, to the parameters corresponding to the instantiated PWMServoObjectTypeDef object.

Function pwm_servo_set_offset:

```
52  void pwm_servo_set_offset(PWMServoObjectTypeDef *self, int offset)
53  {
54      offset = offset < -100 ? -100 : (offset > 100 ? 100 : offset); /* 限制最小/最大偏差，不同舵机极限会不
55      self->offset = offset;
56  }
57
```

This function is used to set the offset of the servo.

Parameters: PWMServoObjectTypeDef instantiated object that needs to be controlled, PWM servo offset.

Here, the magnitude limit of the offset will be set. When the servo rotates, this parameter will be used for adjustment.

Function pwm_servo_object_init:

```
58  void pwm_servo_object_init(PWMServoObjectTypeDef *obj)
59  {
60      memset(obj, 0, sizeof(PWMServoObjectTypeDef));
61      obj->current_duty = 1500; /* 默认位置 */
62      obj->duty_raw = 1500;     /* 默认实际脉宽 */
63  }
```

This function is the initialization function for the PWM servo PWMServoObjectTypeDef instantiated object.

Parameters: PWMServoObjectTypeDef instantiated object that needs to be controlled.

Firstly, new memory is allocated for initialization, and then the initial position of the PWM servo is set.

## 5.3 Initialization

File location: Hiwonder/Porting

Instantiation of PWM servo objects:

```
6   PWMServoObjectTypeDef *pwm_servos[4];
```

Instantiate objects of the PWMServoObjectTypeDef structure for PWM servos 1-4.

Setting servo:

```
static void pwm_servo1_write_pin(uint32_t new_state)
{
    HAL_GPIO_WritePin(PWM_SERVO_1_GPIO_Port, PWM_SERVO_1_Pin, (GPIO_PinState)new_state);
}

static void pwm_servo2_write_pin(uint32_t new_state)
{
    HAL_GPIO_WritePin(PWM_SERVO_2_GPIO_Port, PWM_SERVO_2_Pin, (GPIO_PinState)new_state);
}

static void pwm_servo3_write_pin(uint32_t new_state)
{
    HAL_GPIO_WritePin(PWM_SERVO_3_GPIO_Port, PWM_SERVO_3_Pin, (GPIO_PinState)new_state);
}

static void pwm_servo4_write_pin(uint32_t new_state)
{
    HAL_GPIO_WritePin(PWM_SERVO_4_GPIO_Port, PWM_SERVO_4_Pin, (GPIO_PinState)new_state);
}
```

Here, four functions are used to configure the interfaces of four PWM servos. They call HAL_GPIO_WritePin to set the port number, pin, and high or low level for each PWM servo.

Parameter: 'new_state' represents the current pin's high or low level.

PWM servo initialization:

```
void pwm_servos_init(void)
{
    for(int i = 0; i < 4; ++i) {
      pwm_servos[i] = LWMEM_CCM_MALLOC(sizeof(PWMServoObjectTypeDef));
        pwm_servo_object_init(pwm_servos[i]);  // 初始化PWM舵机对象内存
    }
    pwm_servos[0]->write_pin = pwm_servo1_write_pin;
    pwm_servos[1]->write_pin = pwm_servo2_write_pin;
    pwm_servos[2]->write_pin = pwm_servo3_write_pin;
    pwm_servos[3]->write_pin = pwm_servo4_write_pin;

    __HAL_TIM_SET_COUNTER(&htim13, 0);
    __HAL_TIM_CLEAR_FLAG(&htim13, TIM_FLAG_UPDATE);
    __HAL_TIM_CLEAR_FLAG(&htim13, TIM_FLAG_CC1);
    __HAL_TIM_ENABLE_IT(&htim13, TIM_IT_UPDATE);
    __HAL_TIM_ENABLE_IT(&htim13, TIM_IT_CC1);
    __HAL_TIM_ENABLE(&htim13);
}
```

This function is used for initializing PWM servos.

Firstly, it initializes the memory of existing PWM objects and resets them to their initial positions.

Then, it assigns the function for setting servo levels to the write_pin function of the PWM servo object.

Finally, it sets up the timer.